UNIVERSIDAD TÉCNICA DE AMBATO

Proyecto integrador de la asignatura Estructura de Datos

SMARTCINE WEB

Desarrollo de un aplicativo web para la gestión de catálogo, ventas, alquileres y reservas, aplicando estructuras de datos avanzadas y arquitectura de software.

Carrera	Ingeniería de Software
Asignatura	Estructura de Datos
Tipo de trabajo	Proyecto integrador avanzado
Modalidad	Aplicativo web con base de datos y arquitectura por capas
Contexto	Propuesta alineada con la planificación académica de la asignatura

Docente: Ing. José Caiza

Estudiantes: Acurio Lopez Paul Medardo, Leon Tusa Jair Alexander, Zamora Arroba Alan Santiago

Período académico: Enero-Julio

Fecha: 07/04/2026

1. Análisis

1.1. Introducción

En la formación de Ingeniería de Software, el estudio de estructuras de datos adquiere mayor valor cuando se aplica dentro de soluciones completas y no únicamente como ejercicios aislados de programación. Por ello, este proyecto integrador propone que los estudiantes diseñen y construyan un aplicativo web funcional, partiendo de requerimientos iniciales, diseño de base de datos, arquitectura de software por capas e integración de estructuras de datos lineales y no lineales.

La propuesta responde a la secuencia de contenidos de la asignatura, que aborda TDA, listas, pilas, colas, árboles, recursividad y grafos, y busca que cada equipo resuelva un problema realista de gestión universitaria con criterios técnicos, de calidad y de documentación profesional.

1.2. Justificación

El proyecto permite evaluar de forma articulada la capacidad del estudiante para analizar problemas, levantar requerimientos, modelar datos, diseñar arquitectura de software y seleccionar estructuras de datos apropiadas. Además, fortalece la visión de ingeniería al exigir que cada estructura implementada tenga un uso real dentro del sistema y no sea incorporada únicamente como demostración académica.

- Integra teoría y práctica en un solo producto de software.
- Promueve decisiones técnicas justificadas y documentadas.
- Refuerza competencias de desarrollo web, base de datos y modelado.
- Facilita una evaluación más completa del desempeño del estudiante.

1.3. Problema a resolver

En un videoclub comercial, procesos como la gestión del catálogo de películas, el registro de clientes y el flujo constante de ventas, alquileres, reservas y devoluciones suelen generar retrasos y poca trazabilidad si no se gestionan adecuadamente. Se requiere un sistema que centralice estos procesos y represente la información mediante estructuras de datos dinámicas adecuadas para optimizar las operaciones.

1.4. Tema del proyecto

Desarrollo de un software de administración para un videoclub comercial, aplicando estructuras de datos avanzadas para la gestión eficiente del catálogo, usuarios, reservas y devoluciones.

1.5. Objetivo general

Desarrollar un aplicativo integral para la gestión de atención, trámites y catálogo de un videoclub comercial, aplicando estructuras de datos lineales y no lineales, diseño de base de datos a partir de requerimientos iniciales y una arquitectura de software por capas.

1.6. Objetivos específicos

- Levantar y documentar los requerimientos funcionales y no funcionales del sistema.
- Analizar el problema y modelar los procesos principales del aplicativo.
- Diseñar la base de datos desde los requerimientos iniciales, incluyendo modelo conceptual, lógico y físico.
- Implementar estructuras de datos propias y asociarlas a funcionalidades reales del sistema.
- Diseñar una arquitectura de software web organizada por capas.
- Desarrollar el aplicativo web con interfaz, lógica de negocio, persistencia y seguridad básica.
- Validar el funcionamiento del sistema mediante pruebas funcionales y técnicas.
- Documentar y defender las decisiones tomadas en el proyecto.

1.7. Alcance del proyecto

El sistema deberá incluir, como mínimo, la autenticación de usuarios y la administración de roles básicos (Administrador, Empleado, Cliente), así como el registro de usuarios del videoclub. A nivel operativo, abarcará la gestión del inventario principal de películas, el procesamiento de ventas definitivas y alquileres, la atención de devoluciones rápidas y la gestión de reservas para títulos de alta demanda.

Además, el alcance contempla la visualización del historial cronológico de transacciones por usuario, la exhibición continua de la cartelera de estrenos, la organización jerárquica del catálogo cinematográfico (categorías y géneros) y un motor interno de similitudes para recomendaciones de películas. Toda la solución contará con almacenamiento persistente en una base de datos relacionales y se entregará mediante una interfaz web funcional estructurada bajo una arquitectura por capas.

1.8. Requerimientos iniciales

1.8.1. Requerimientos funcionales

Código	Descripción
RF1	Registrar usuarios y gestionar la autenticación del sistema, administrando roles y permisos básicos (Administrador, Empleado, Cliente).
RF2	Gestionar el inventario principal mediante la inserción de nuevos títulos y la baja definitiva de copias (eliminación de nodos) por venta directa o daño físico.
RF3	Registrar clientes y gestionar su historial dinámico de transacciones, permitiendo la navegación bidireccional cronológica para diferenciar entre alquileres y compras.
RF4	Exhibir de forma continua las películas destacadas o estrenos en un bucle infinito para la interfaz visual del sistema.
RF5	Procesar la venta directa de películas, registrando el ingreso económico y descontando permanentemente el artículo del inventario.
RF6	Gestionar el retorno de películas simulando un Buzón de Devoluciones Rápidas, donde el empleado procesa el último elemento depositado básicamente en la cima de la pila (LIFO).
RF7	Gestionar las reservas de películas de alta demanda o en liquidación, asegurando que la asignación respeta estrictamente el orden de llegada de las solicitudes (FIFO).
RF8	Organizar el catálogo de películas mediante una clasificación jerárquica (Categoría > Género > Subgénero) para optimizar la búsqueda y el filtrado.

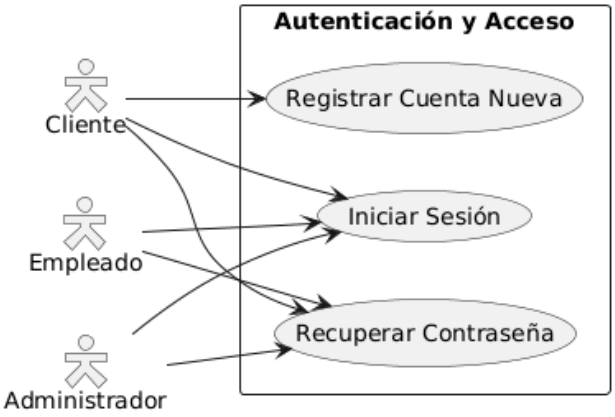
RF9	Implementar un motor de búsqueda de rutas o mapa de similitudes que conecte películas basadas en actores, directores o preferencias conjuntas de otros clientes para generar recomendaciones.
RF10	Calcular y aplicar automáticamente recargas en el perfil del cliente cuando la fecha límite de un alquiler haya expirado.
RF11	Generar reportes básicos y una bitácora de acciones que detalle ingresos diarios, alquileres activos y auditoría del sistema

1.8.2. Requerimientos no funcionales

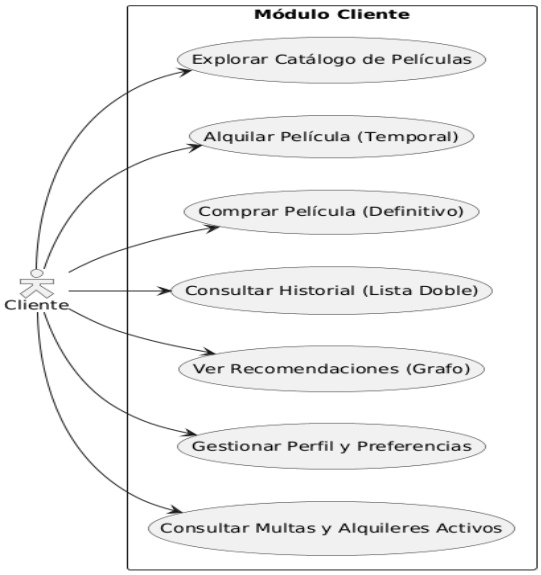
- El sistema debe implementarse como aplicativo web.
- Debe usar una base de datos relacional con integridad referencial.
- La solución debe organizarse con arquitectura por capas.
- Debe incluir validación de formularios y manejo básico de errores.
- La interfaz debe ser usable, consistente y claramente navegable.
- Debe existir documentación técnica y justificación del uso de estructuras de datos

1.9. Creación de Casos de uso

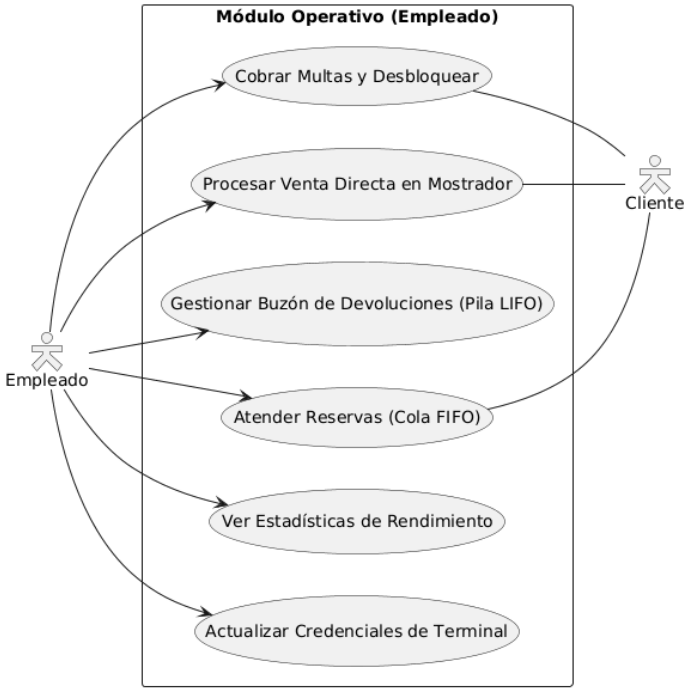
1.9.1. Modulo: Autenticacion y Acceso



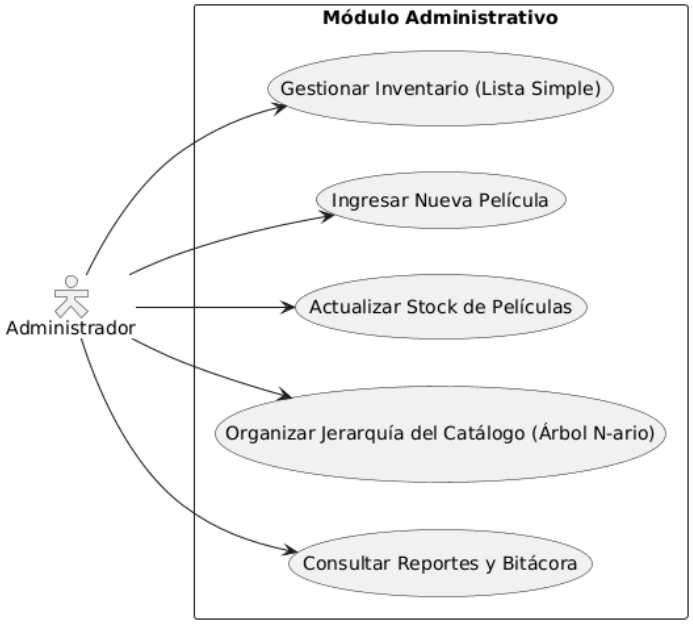
1.9.2 Módulo: Cliente



1.9.3 Módulo: Operativo (Empleado)

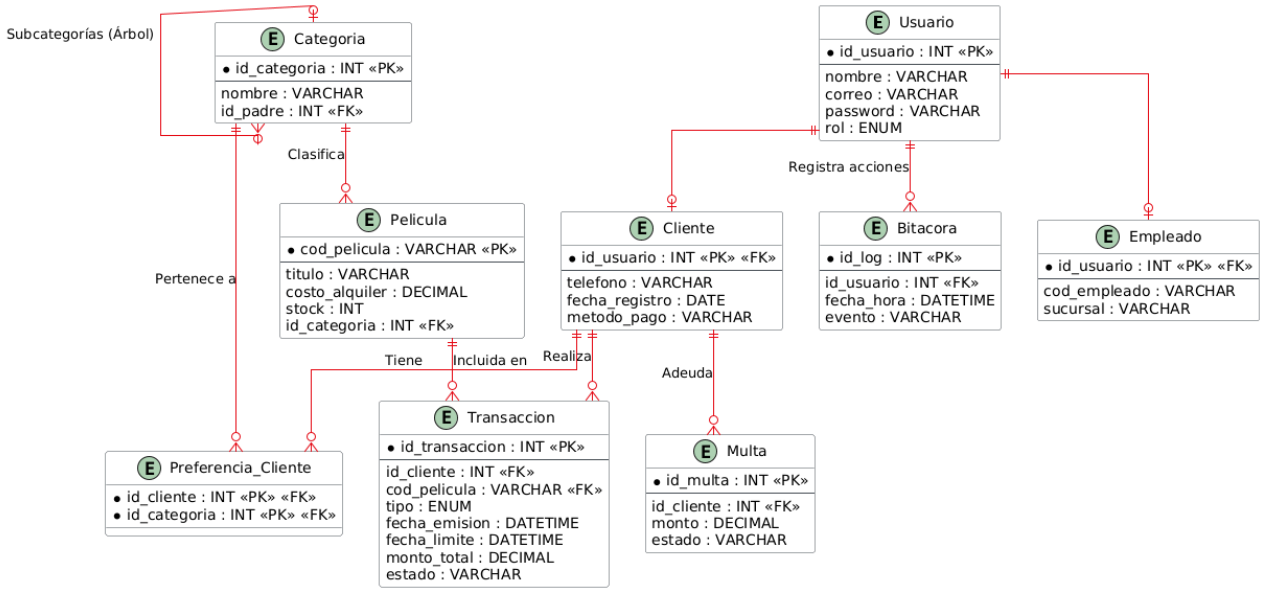


1.9.4 Modulo: Administrativo



2. Base de Datos

2.1. MER (Modelo Entidad-Relación)



2.2. Modelo Lógico Relacional

- **Usuario** (Superclase para el login)
  - id_usuario (PK, INT)
  - nombre (VARCHAR)
  - correo (VARCHAR, UNIQUE)
  - password (VARCHAR)
  - rol (ENUM: 'Cliente', 'Empleado', 'Admin')
- **Cliente** (Subclase)
  - id_usuario (PK, FK -> Usuario.id_usuario)
  - telefono (VARCHAR)
  - fecha_registro (DATE)
  - metodo_pago (VARCHAR)
- **Empleado** (Subclase)
  - id_usuario (PK, FK -> Usuario.id_usuario)

- cod_empleado (VARCHAR, UNIQUE)
- sucursal (VARCHAR)

➢ **Categoria** (Soporta el Árbol N-ario)

- id_categoria (PK, INT)
- nombre (VARCHAR)
- id_padre (FK -> Categoria.id_categoria, NULLable)

➢ **Pelicula** (Inventario)

- cod_pelicula (PK, VARCHAR)
- titulo (VARCHAR)
- id_categoria (FK -> Categoria.id_categoria)
- costo_alquiler (DECIMAL)
- stock (INT)

➢ **Transaccion** (Alquileres, Compras y Mostrador)

- id_transaccion (PK, INT)
- id_cliente (FK -> Cliente.id_usuario, NULLable para ventas anónimas en mostrador)
- cod_pelicula (FK -> Pelicula.cod_pelicula)
- tipo (ENUM: 'Alquiler', 'Compra')
- fecha_emision (DATETIME)
- fecha_limite (DATETIME, NULLable)
- monto_total (DECIMAL)
- estado (ENUM: 'Activo', 'Devuelto', 'Vendido')

➢ **Multa**

- id_multa (PK, INT)
- id_cliente (FK -> Cliente.id_usuario)
- monto (DECIMAL)
- estado (ENUM: 'Pendiente', 'Pagada')

- **Preferencia_Cliente** (Grafo de gustos)
    - id_cliente (PK, FK -> Cliente.id_usuario)
    - id_categoria (PK, FK -> Categoria.id_categoria)
- **Bitacora** (Auditoría)
    - id_log (PK, INT)
    - id_usuario (FK -> Usuario.id_usuario)
    - fecha_hora (DATETIME)
    - evento (VARCHAR)

2.3. Proceso de Normalización

- **1NF (Primera Forma Normal - Atómicos):** En el perfil del cliente, los géneros favoritos estaban en una lista (Ciencia Ficción, Acción, Drama). Para cumplir la 1NF, separamos esto en una tabla intermedia (Preferencia_Cliente) para evitar atributos multivaluados.
- **2NF (Segunda Forma Normal - Dependencias Parciales):** Aseguramos que tablas con claves compuestas dependan de toda la clave. Por ejemplo, en la tabla intermedia de preferencias, tanto el Cliente como la Categoría son necesarios.
- **3NF (Tercera Forma Normal - Dependencias Transitivas):** En el inventario de películas, el "Género" no se guarda como texto dentro de la tabla de películas. Se extrae a una tabla Categoria (que además nos sirve para armar el Árbol N-ario del RF8) y se vincula mediante una clave foránea, evitando redundancia de datos.

2.4. Script SQL

Este Script SQL está diseñado para PostgreSQL.

EXPLICACIÓN

```sql
-- 1. Tabla Categoria
CREATE TABLE Categoria (
    id_categoria SERIAL PRIMARY KEY,
    nombre VARCHAR(50) NOT NULL,
    id_padre INT REFERENCES Categoria(id_categoria)
);
```

Este bloque crea la lógica para organizar las películas por categorías y subcategorías.

- **SERIAL PRIMARY KEY**: Crea un número único automático para cada categoría.

- **id_padre INT REFERENCES Categoria**: Al apuntar a la misma tabla, permites que una categoría (ej. "Terror") tenga una subcategoría (ej. "Zombies"). Esto forma el Árbol N-ario.

```sql
-- 2. Tabla Usuario
CREATE TABLE Usuario (
    id_usuario SERIAL PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    correo VARCHAR(100) UNIQUE NOT NULL,
    password VARCHAR(255) NOT NULL,
    rol VARCHAR(20) CHECK (rol IN ('Cliente', 'Empleado', 'Admin'))
);
```

Define quién puede entrar a la aplicación y qué puede hacer.

- **UNIQUE**: Asegura que nadie se registre dos veces con el mismo correo.
- **CHECK (rol IN...)**: Es una regla de validación. Solo permite que el usuario tenga uno de esos tres niveles de acceso.

```sql
-- 3. Tabla Cliente
CREATE TABLE Cliente (
    id_usuario INT PRIMARY KEY REFERENCES Usuario(id_usuario) ON DELETE CASCADE,
    telefono VARCHAR(15),
    fecha_registro DATE DEFAULT CURRENT_DATE,
    metodo_pago VARCHAR(50)
);
```

Aquí se guarda la información específica.

- **ON DELETE CASCADE**: Si borras a un usuario, automáticamente se borran sus datos de cliente.
- **DEFAULT CURRENT_DATE**: El sistema pone automáticamente la fecha en que se registró la persona.

```sql
-- 4. Tabla Pelicula
CREATE TABLE Pelicula (
    cod_pelicula VARCHAR(20) PRIMARY KEY,
    titulo VARCHAR(150) NOT NULL,
    id_categoria INT REFERENCES Categoria(id_categoria),
    costo_alquiler DECIMAL(10,2),
    stock INT DEFAULT 0
);
```

Es la base, donde se muestran las películas disponibles.

- **cod_pelicula**: Usamos un código manual (como un ISBN o código de barras) como clave principal.

- **DECIMAL(10,2)**: Formato numérico para dinero (ej. 4.50), evitando errores de redondeo que dan los tipos "float".

```sql
-- 5. Tabla Transaccion
CREATE TABLE Transaccion (
    id_transaccion SERIAL PRIMARY KEY,
    id_cliente INT REFERENCES Cliente(id_usuario),
    cod_pelicula VARCHAR(20) REFERENCES Pelicula(cod_pelicula),
    tipo VARCHAR(20) CHECK (tipo IN ('Alquiler', 'Compra')),
    fecha_emision TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    monto_total DECIMAL(10,2),
    estado VARCHAR(20)
);
```

Registra cada vez que alguien alquila o compra.

- **TIMESTAMP**: Guarda la fecha y la hora exacta del alquiler.
- **tipo**: Diferencia si el usuario se llevó la película para siempre o debe devolverla.

```sql
-- 6. Tabla Preferencia_Cliente
CREATE TABLE Preferencia_Cliente (
    id_cliente INT REFERENCES Cliente(id_usuario),
    id_categoria INT REFERENCES Categoria(id_categoria),
    PRIMARY KEY (id_cliente, id_categoria)
);
```

Esta tabla es la que cumple con la 1NF (Primera Forma Normal) y el requisito de Grafos.

- **Clave Primaria Compuesta**: Al usar los dos IDs como llave, el sistema sabe que el Cliente A está conectado con la Categoría B.

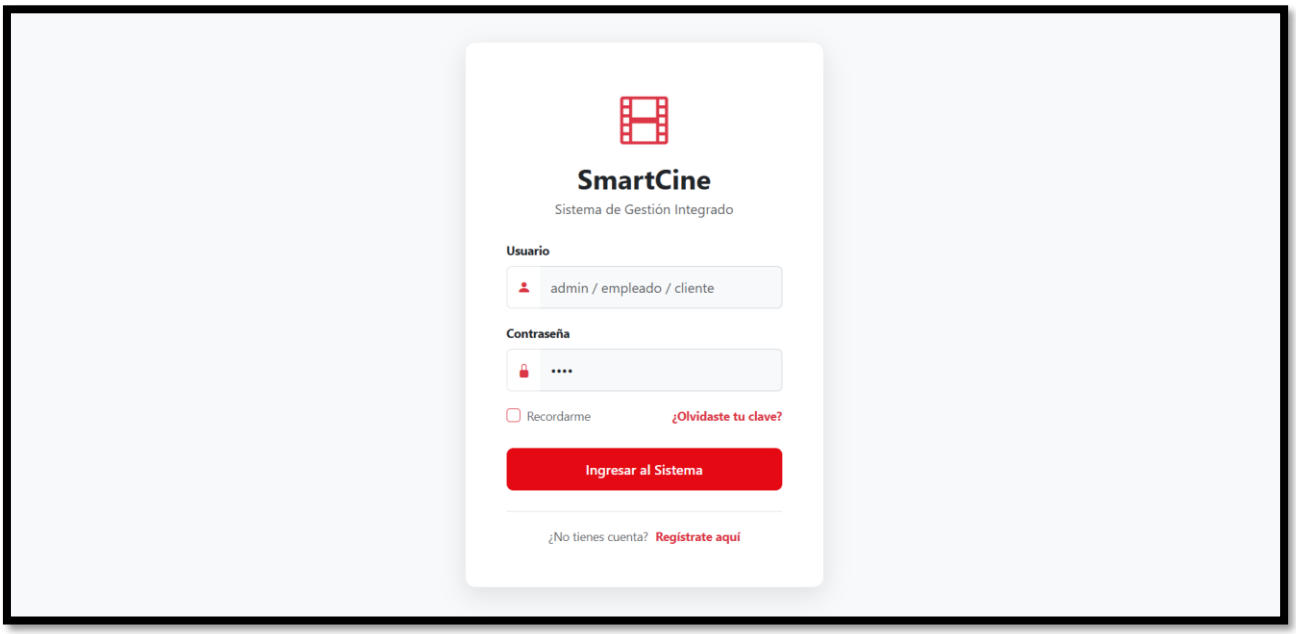
```sql
-- 7. Tabla Multa
CREATE TABLE Multa (
    id_multa SERIAL PRIMARY KEY,
    id_cliente INT REFERENCES Cliente(id_usuario),
    monto DECIMAL(10,2),
    estado VARCHAR(20) DEFAULT 'Pendiente'
);

-- 8. Tabla Bitacora
CREATE TABLE Bitacora (
    id_log SERIAL PRIMARY KEY,
    id_usuario INT REFERENCES Usuario(id_usuario),
    fecha_hora TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    evento VARCHAR(255)
);
```

Asegura que el sistema sea profesional y seguro.

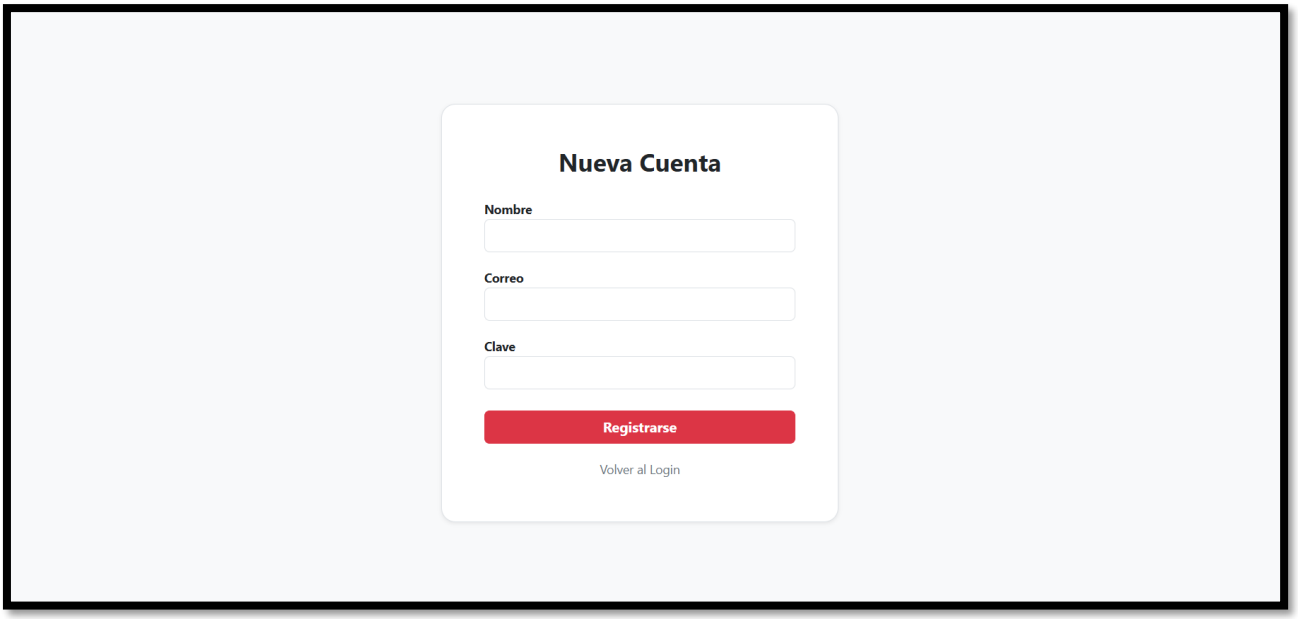
- **Multa**: Se activa si un alquiler de la tabla Transaccion se pasa de la fecha.
- **Bitacora**: Registra cada movimiento (ej. "El admin borró una película") para que siempre haya un historial de seguridad.

3. Prototipo inicial

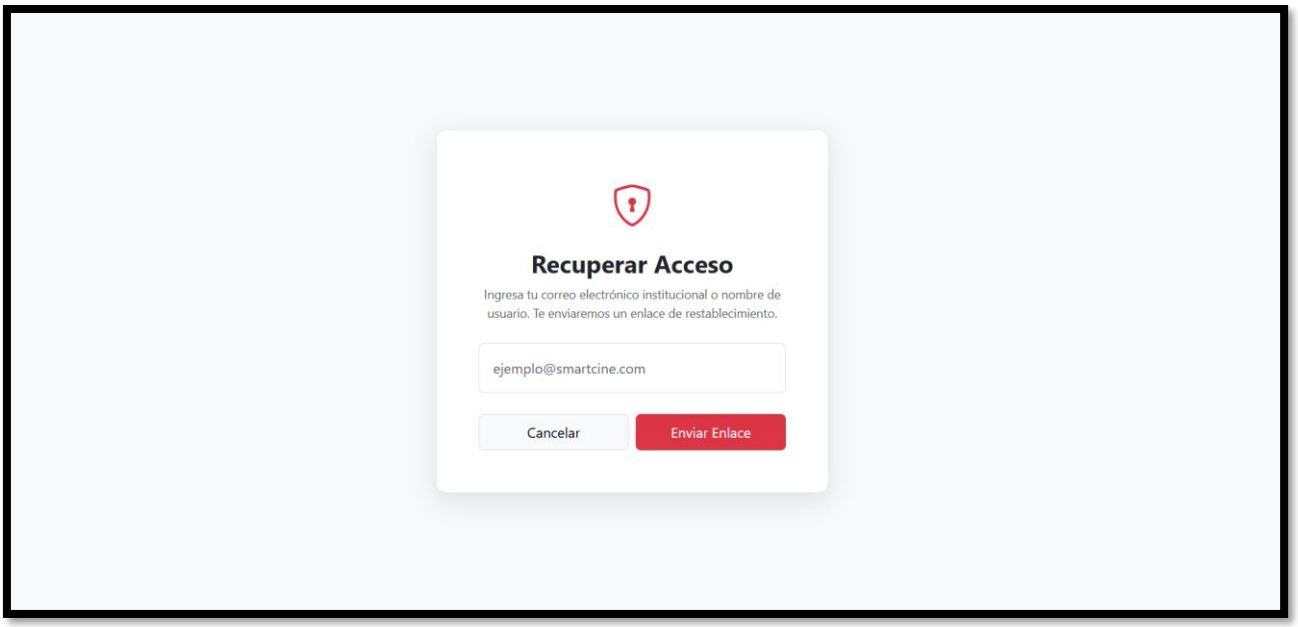
3.1. Inicio Sesión



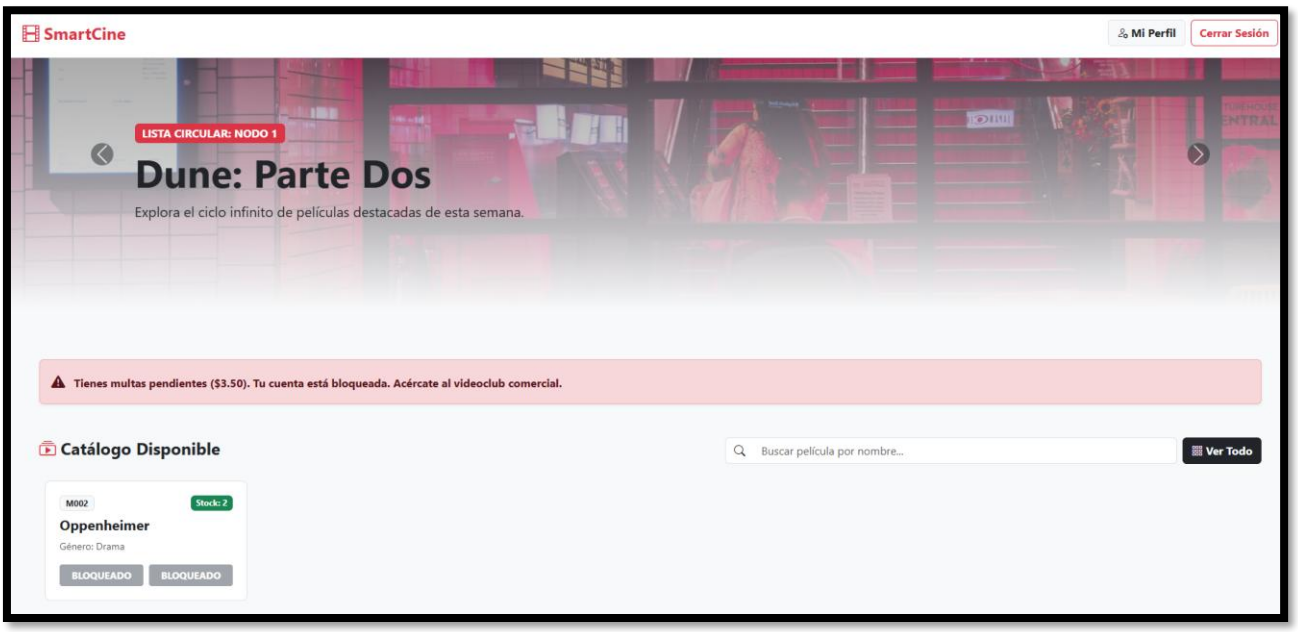
3.2. Registrar Usuario



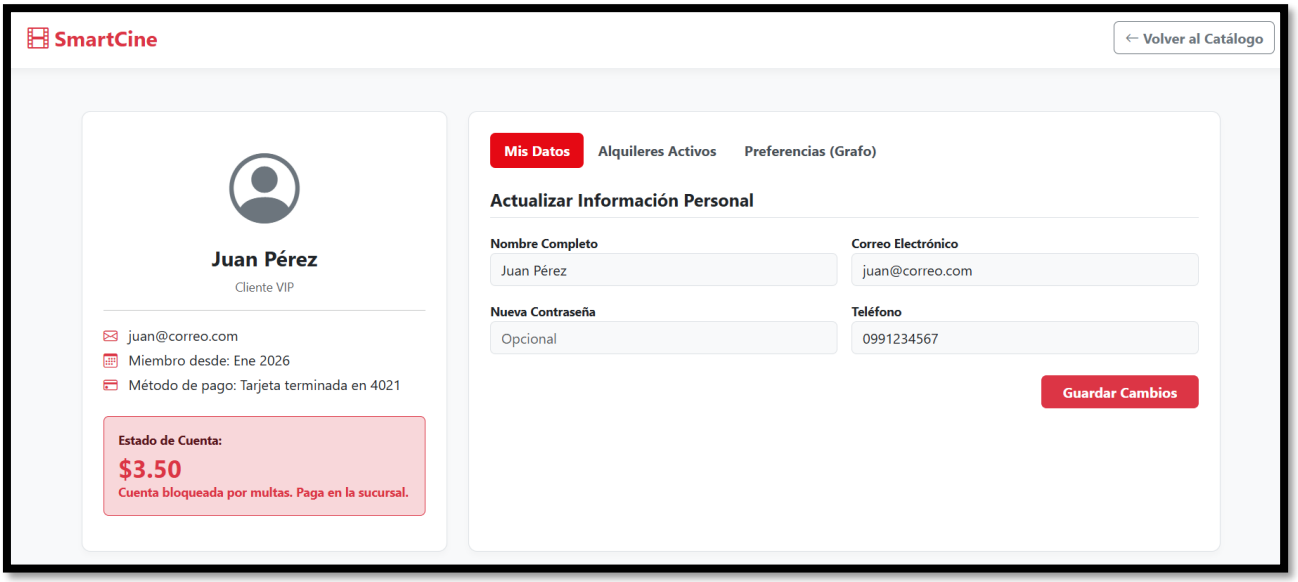
3.3. Recuperar contraseña

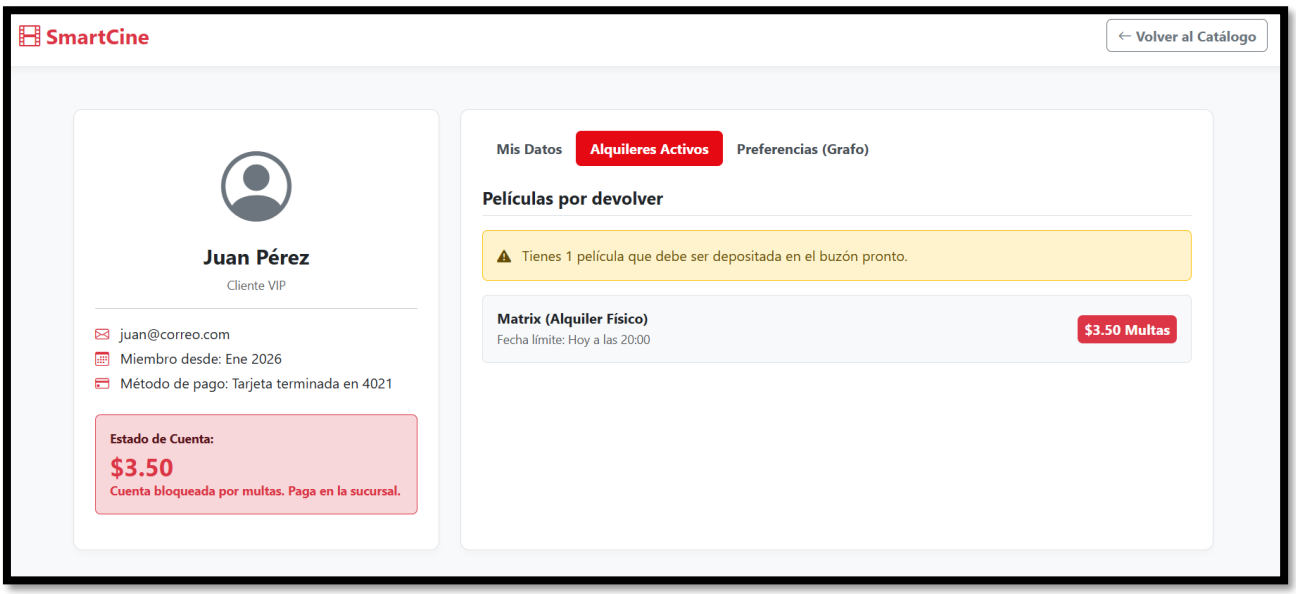


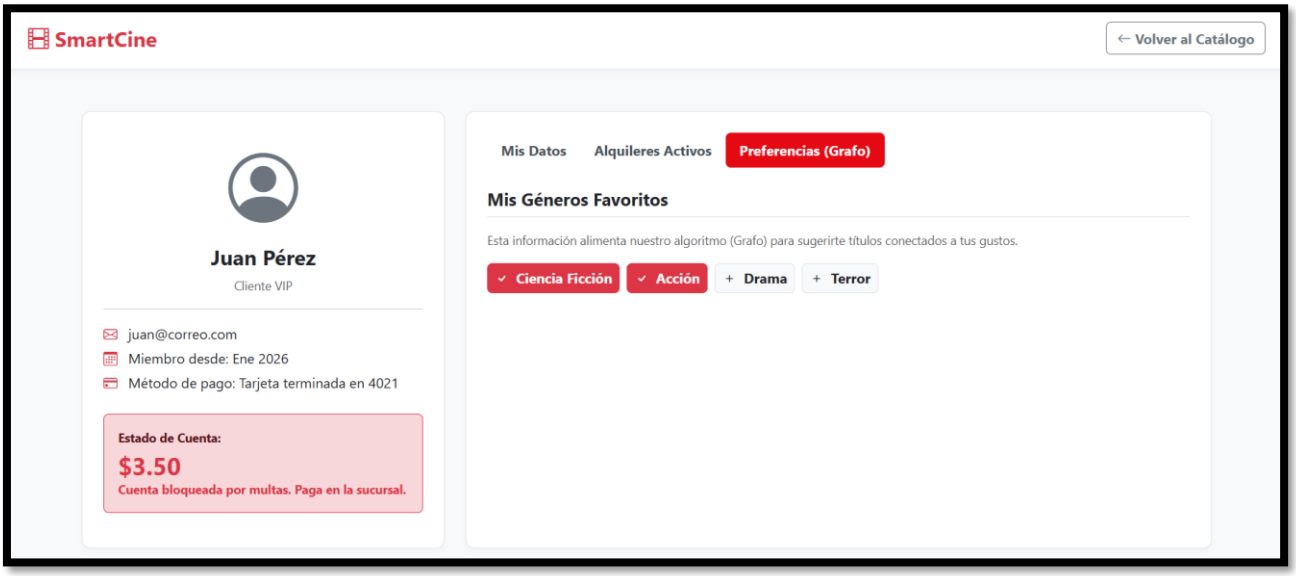
3.4. Vista Cliente



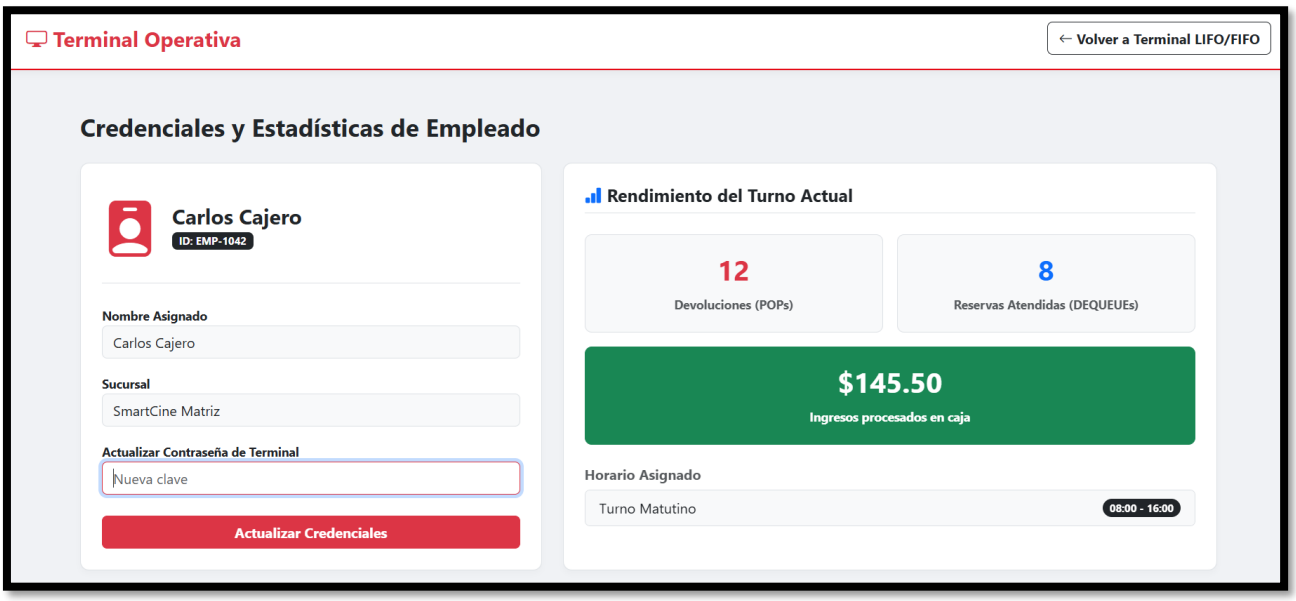
3.5. Vista Perfil Cliente



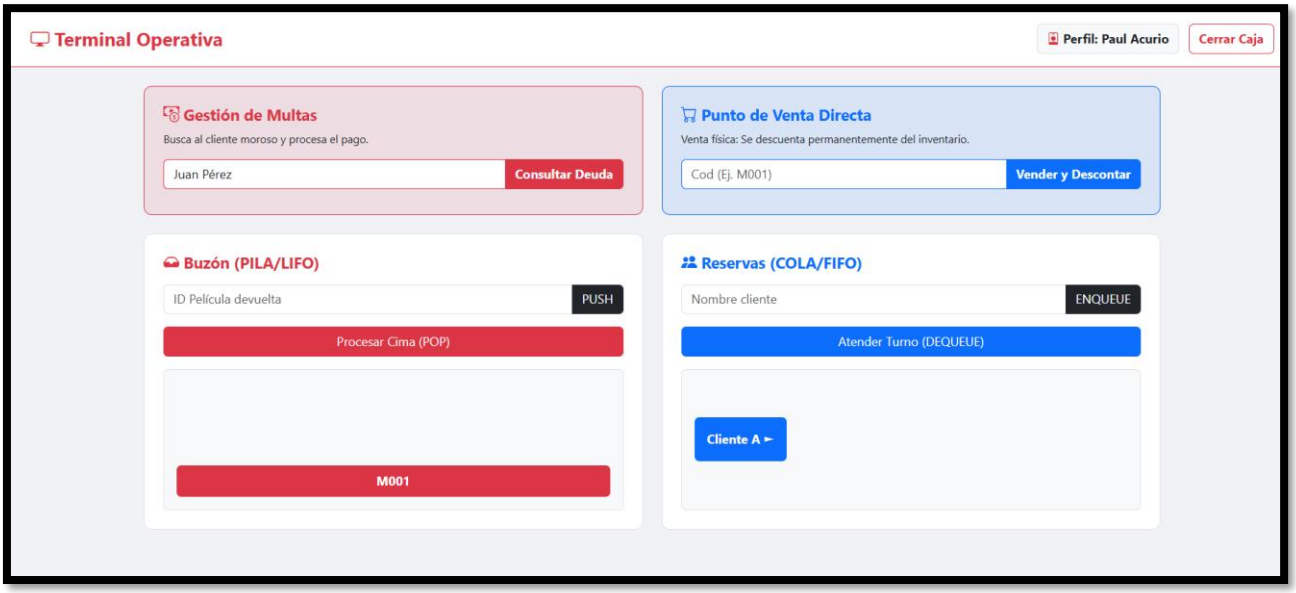




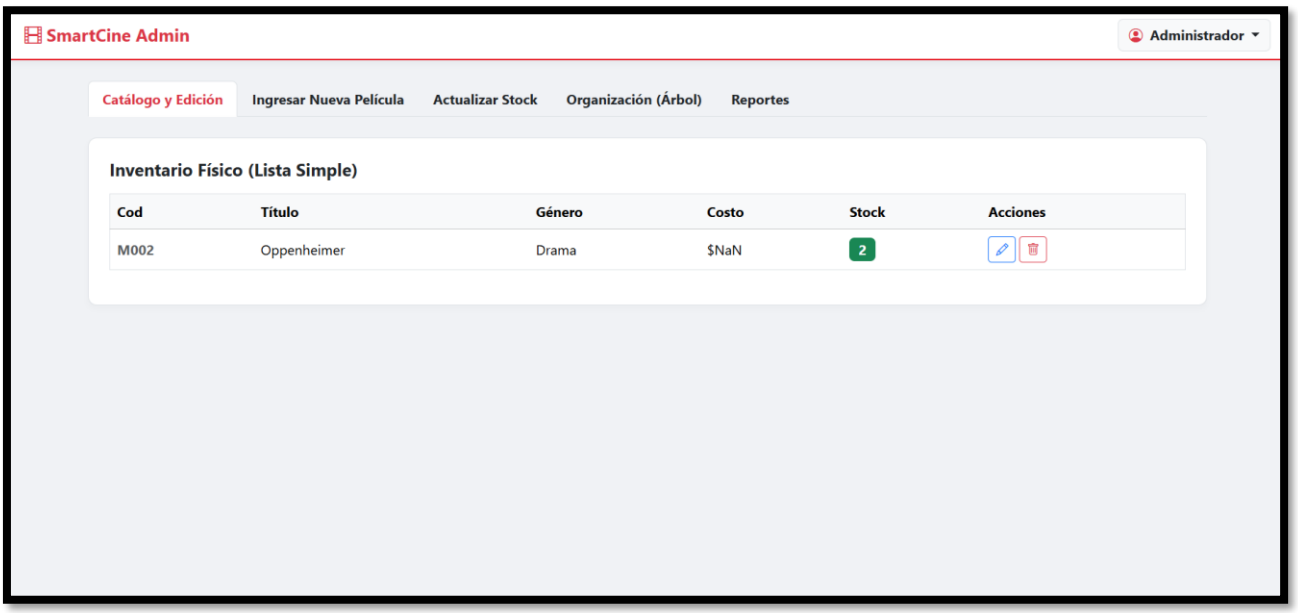
3.6. Vista Perfil Empleado

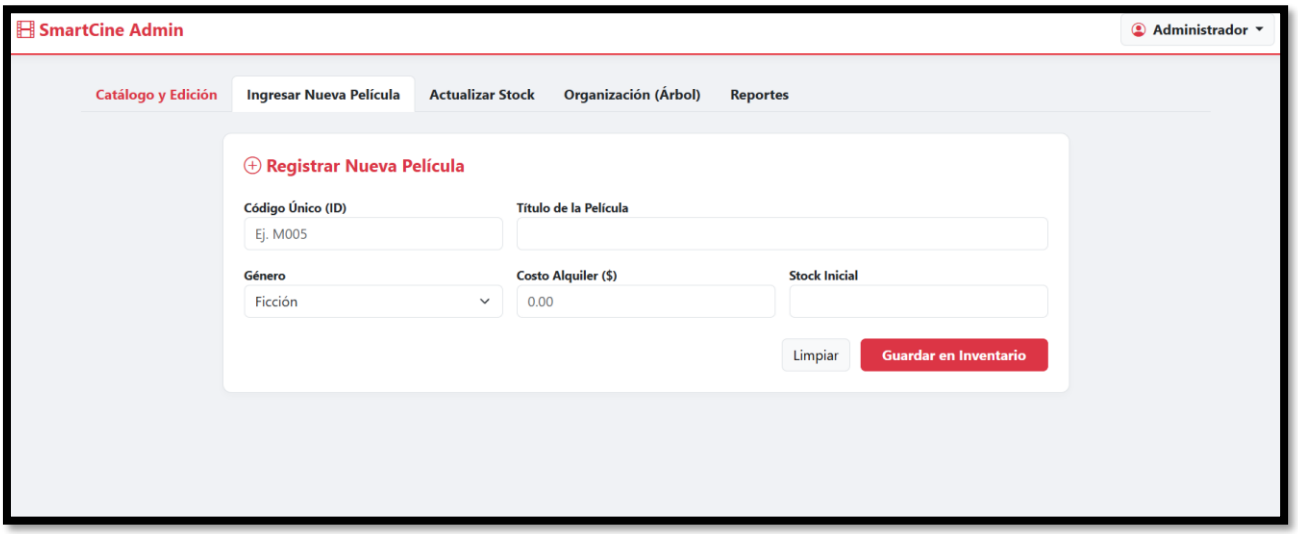


3.7. Vista Empleado

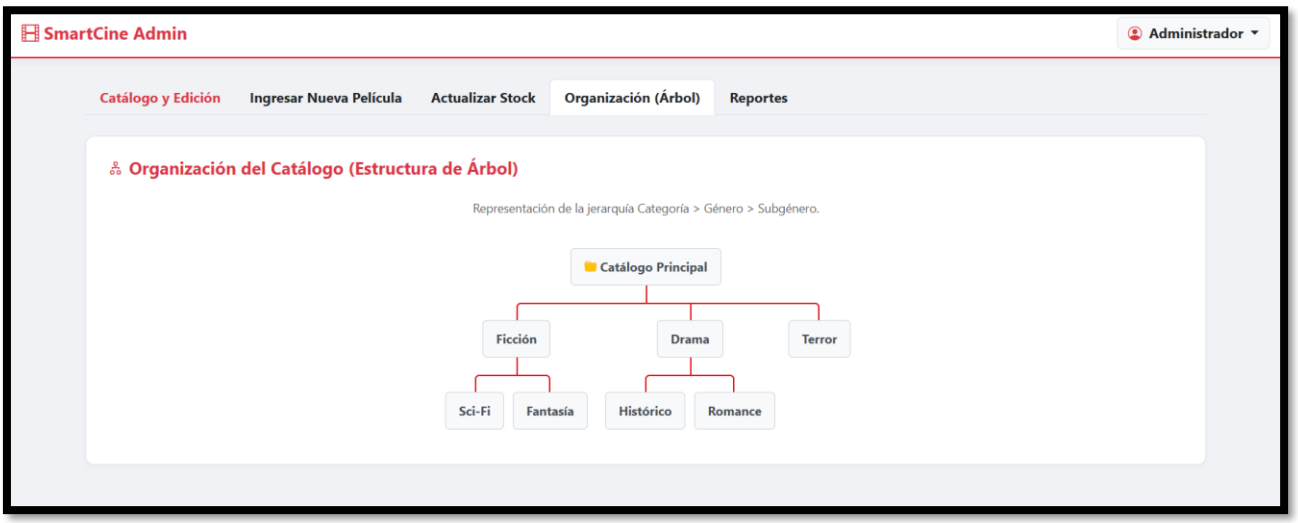


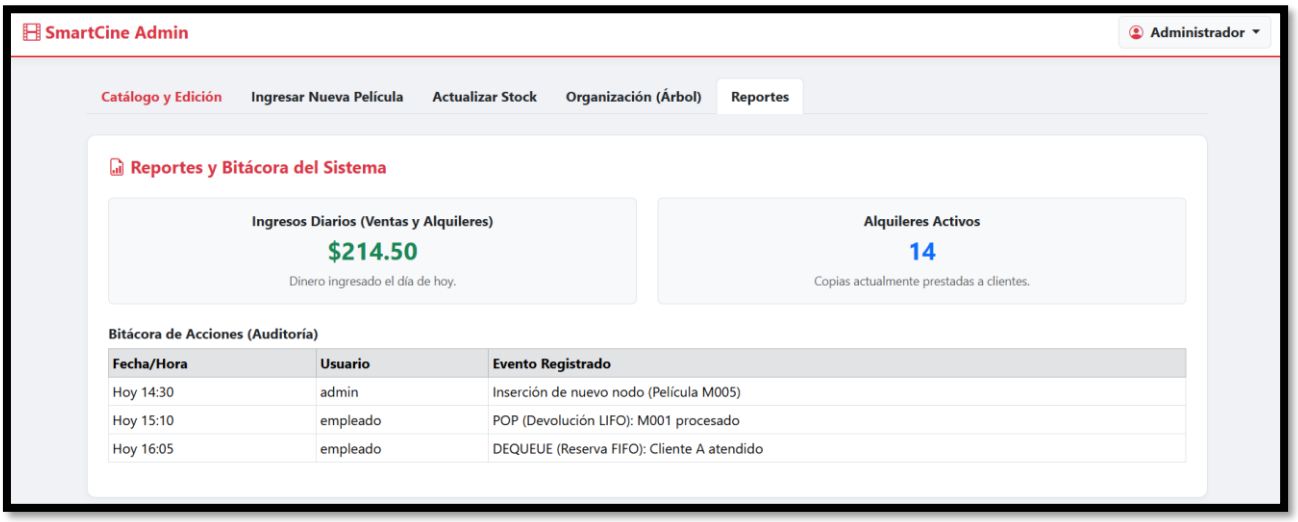
3.8. Vista Admin


SmartCine Admin
Administrador
Catálogo y Edición
Ingresar Nueva Película
Actualizar Stock
Organización (Árbol)
Reportes
Inventario Físico (Lista Simple)
Cod
Título
Género
Costo
Stock
Acciones
M002
Oppenheimer
Drama
$NaN
2


SmartCine Admin
Administrador
Catálogo y Edición
Ingresar Nueva Película
Actualizar Stock
Organización (Árbol)
Reportes
Registrar Nueva Película
Código Único (ID)
Ej. M005
Título de la Película
Género
Ficción
Costo Alquiler ($)
0.00
Stock Inicial
Limpiar
Guardar en Inventario

SmartCine Admin
Administrador
Catálogo y Edición
Ingresar Nueva Película
Actualizar Stock
Organización (Árbol)
Reportes
Ingreso de Stock a Existentes
Ingrese el código de la película para buscarla en la Lista y sumar unidades.
Código (Ej. M001)
Buscar


SmartCine Admin
Administrador
Catálogo y Edición
Ingresar Nueva Película
Actualizar Stock
Organización (Árbol)
Reportes
Organización del Catálogo (Estructura de Árbol)
Representación de la jerarquía Categoría > Género > Subgénero.
Catálogo Principal
Ficción
Drama
Terror
Sci-Fi
Fantasía
Histórico
Romance


SmartCine Admin
Administrador
Catálogo y Edición
Ingresar Nueva Película
Actualizar Stock
Organización (Árbol)
Reportes
Reportes y Bitácora del Sistema
Ingresos Diarios (Ventas y Alquileres)
$214.50
Dinero ingresado el día de hoy.
Alquileres Activos
14
Copias actualmente prestadas a clientes.
Bitácora de Acciones (Auditoría)
Fecha/Hora
Usuario
Evento Registrado
Hoy 14:30
admin
Inserción de nuevo nodo (Película M005)
Hoy 15:10
empleado
POP (Devolución LIFO): M001 procesado
Hoy 16:05
empleado
DEQUEUE (Reserva FIFO): Cliente A atendido